



IT Academy
by KIBERNUM

Funciones Flecha





Funciones de Flecha vs. Funciones Tradicionales en JavaScript

En JavaScript hay **dos formas principales** de escribir funciones:

- ✓ Funciones tradicionales (function)
- ✓ Funciones de flecha (arrow functions, `=>`)

◆ [1] ¿Qué es una función tradicional?

Es la forma "**clásica**" de escribir una función en JavaScript.

📌 Ejemplo básico:

```
function saludar(nombre) {  
  return `Hola, ${nombre}!`;  
}  
  
console.log(saludar("Juan")); // Hola, Juan!
```

◆ [2] ¿Qué es una función de flecha (=>)?

Es una forma más corta y moderna de escribir funciones. Se usa el símbolo `=>` en lugar de la palabra function.

📌 El mismo ejemplo con función de flecha:

```
const saludar = (nombre) => `Hola, ${nombre}!`;  
  
console.log(saludar("Juan")); // Hola, Juan!
```

Cómo transformar una función tradicional a función de flecha (=>) en JavaScript

Las funciones de flecha (=>) son una forma más corta y moderna de escribir funciones en JavaScript. Vamos a aprender cómo convertir una función tradicional en una función de flecha paso a paso.

Función tradicional:

```
function saludar() {  
  return "¡Hola, bienvenido!";  
}  
console.log(saludar());
```

Función de flecha (=>):

```
const saludar = () => "¡Hola, bienvenido!";  
console.log(saludar());
```

Reglas básicas de transformación

- 1 Quitar la palabra function y agregar => después de los parámetros.
- 2 Si la función solo tiene una línea de código, puedes quitar {} y return.
- 3 Si hay un solo parámetro, puedes quitar los paréntesis ().

Explicación:

- Quitamos **function** y agregamos => después de ().
- Como la función solo tiene una línea, eliminamos {} y **return**.

Reto Transformar Funciones



Transformando Funciones en JavaScript

Las funciones de flecha (**=>**) son una forma más corta y moderna de escribir funciones en JavaScript. Vamos a aprender cómo convertir una función tradicional en una función de flecha paso a paso.

 **Objetivo:** Los estudiantes explorarán la transformación de funciones tradicionales en funciones de flecha (**=>**) aplicando las reglas clave de simplificación en JavaScript.

◆ **1** Analiza la función tradicional

```
function calcularDoble(numero) {  
  return numero * 2;  
}
```

```
console.log(calcularDoble(5));  
console.log(calcularDoble(8));
```

Ejecuta el siguiente código en la consola de tu navegador y observa el resultado

Pregunta para reflexionar:

- ¿Qué hace esta función?
- ¿Cómo podrías simplificarla usando una función de flecha (**=>**)?

 Toma una captura de pantalla de la ejecución en la consola.

Pregunta

Después de aprender sobre funciones en JavaScript, piensa en lo siguiente:

Si tuvieras que explicarle a alguien qué es una función y por qué es útil,
¿cómo lo harías con un ejemplo de la vida real?

 Pista: Piensa en acciones cotidianas que se repiten, como hacer una receta de cocina o encender una luz con un interruptor. ¿Cómo se relaciona esto con las funciones en JavaScript?

Conclusión

Hemos aprendido que las funciones en JavaScript nos permiten reutilizar código, mejorar la organización y hacer que nuestro programa sea más eficiente. Comprendimos cómo definir funciones, usar parámetros y valores de retorno, y la diferencia entre variables locales y globales.

También exploramos el impacto del alcance de las variables y cómo las funciones anidadas pueden ayudarnos a estructurar mejor nuestro código.

✨ Escribir código limpio y modular con funciones bien diseñadas es clave para desarrollar aplicaciones escalables y mantenibles. ¡Ahora es tu turno de ponerlo en práctica! 🚀





IT Academy

by KIBERNUM